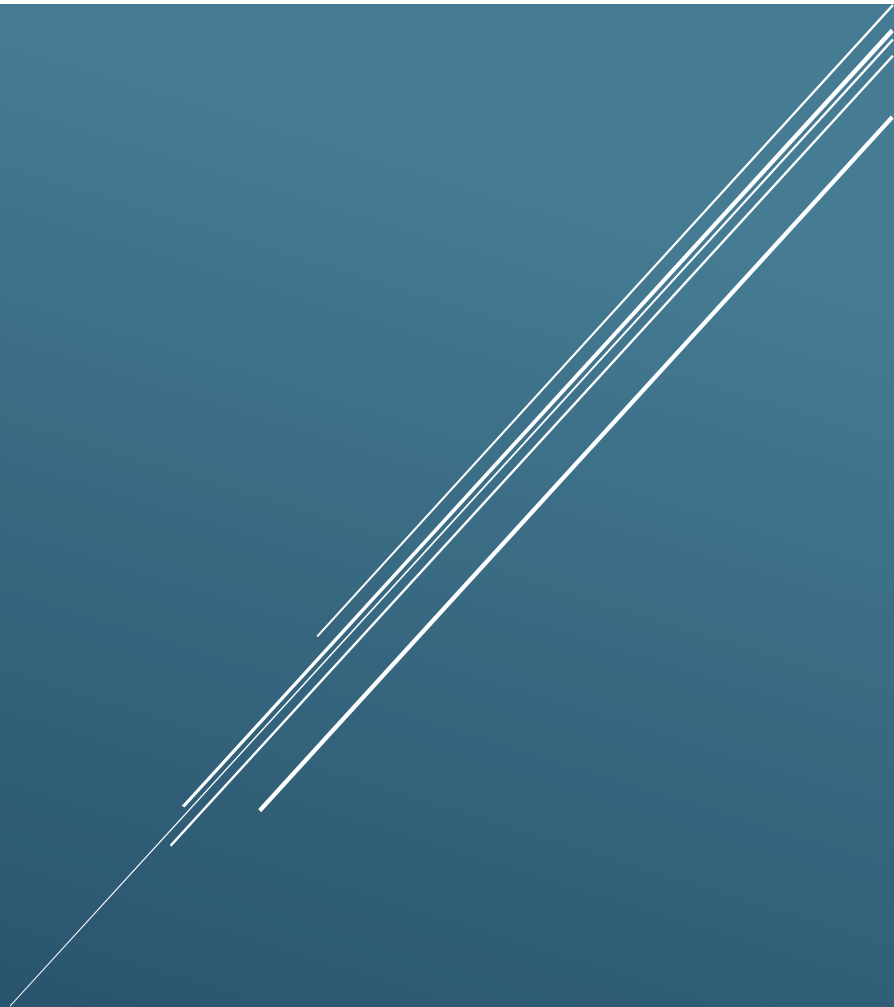




# PRACTICA 6 AWS KUBERNETES

Luis Peregrina Ojeda 2ºASIR Caja Mágica

# Parte 0 Instalamos Kubernetes

Empezamos la practica instalando Kubernetes

Despues de actualizar el sistema (lo primero). Comenzamos la instalacion de kubernetes

```
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ curl -sL https://get.k3s.io | K3S_KUBECONFIG
_MODE="644" sh -
[INFO] Finding release for channel stable
[INFO] Using v1.34.3+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.34.3+k3s1/sha256sum-amd64.
txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.34.3+k3s1/k3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Skipping installation of SELinux RPM
[INFO] Creating /usr/local/bin/kubectl symlink to k3s
[INFO] Creating /usr/local/bin/crictl symlink to k3s
[INFO] Creating /usr/local/bin/ctr symlink to k3s
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s.service
[INFO] systemd: Enabling k3s unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service → /etc/systemd/system/k3s.serv
ice.
[INFO] Host iptables-save/iptables-restore tools not found
[INFO] Host ip6tables-save/ip6tables-restore tools not found
[INFO] systemd: Starting k3s
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo k3s --version
k3s version v1.34.3+k3s1 (48ffa7b6)
go version go1.24.11
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo k3s server & sleep 10
```

Despues de instalarlo me ha dado varios errores y he tenido que hacer varios cambios pero ya funciona...

```
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get nodes
WARN[0000] Unable to read /etc/rancher/k3s/k3s.yaml, please start server with --write-kubeconfig-mode
or --write-kubeconfig-group to modify kube config permissions
error: error loading config file "/etc/rancher/k3s/k3s.yaml": open /etc/rancher/k3s/k3s.yaml: permissi
on denied
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get pods -n kube-system
WARN[0000] Unable to read /etc/rancher/k3s/k3s.yaml, please start server with --write-kubeconfig-mode
or --write-kubeconfig-group to modify kube config permissions
error: error loading config file "/etc/rancher/k3s/k3s.yaml": open /etc/rancher/k3s/k3s.yaml: permissi
on denied
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ ls -l ~/.kube/config
-rw-r----- 1 alumno02 alumno02 2945 Jan 27 09:21 /home/alumno02/.kube/config
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl --kubeconfig=$HOME/.kube/config get n
odes
NAME                STATUS    ROLES          AGE   VERSION
a6alumno02          Ready    control-plane   52m   v1.34.3+k3s1
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ echo 'export KUBECONFIG=$HOME/.kube/config' >
> ~/.bashrc
source ~/.bashrc
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ echo 'export KUBECONFIG=$HOME/.kube/config' >
> ~/.bashrc
source ~/.bashrc
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ source ~/.bashrc
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get nodes
NAME                STATUS    ROLES          AGE   VERSION
a6alumno02          Ready    control-plane   56m   v1.34.3+k3s1
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get pods -n kube-system
NAME                                READY    STATUS    RESTARTS   AGE
coredns-7f496c8d7d-qzj44            1/1      Running   0           56m
helm-install-traefik-crd-j2fcm       0/1      Completed 0           56m
helm-install-traefik-r65xt           0/1      Completed 2           56m
local-path-provisioner-578895bd58-c5tlc 1/1      Running   0           56m
metrics-server-7b9c9c4b9c-bnk6f      1/1      Running   0           56m
svclb-traefik-3366efbe-kv6nv         2/2      Running   0           56m
traefik-6f5f87584-2kdbf             1/1      Running   0           56m
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo k3s kubectl get nodes
[sudo] password for alumno02:
NAME                STATUS    ROLES          AGE   VERSION
a6alumno02          Ready    control-plane   56m   v1.34.3+k3s1
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$
```

# Parte 1 Creamos la aplicación para kubernetes

Instalamos kubectl y comprobamos que funciona, tambien creamos el directorio de trabajo

```
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
100  138  100  138    0     0   468      0  --:--:-- --:--:-- --:--:--   467
100 55.8M  100 55.8M    0     0  47.5M      0  0:00:01 0:00:01 --:--:--  47.5M
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo chmod +x kubectl
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo mv kubectl /usr/local/bin/
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl version --client
Client Version: v1.35.0
Kustomize Version: v5.7.1
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ mkdir -p $HOME/.kube
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo cp /etc/rancher/k3s/k3s.yaml $HOME/.kube/config
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo chmod 600 $HOME/.kube/config
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
a6alumno02    Ready     control-plane  68m   v1.34.3+k3s1
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ mkdir -p ~/kubernetes-aws-practice
alumno02@A6Alumno02: /mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ cd ~/kubernetes-aws-practice
alumno02@A6Alumno02: ~/kubernetes-aws-practice$
```

## Parte 2 Configuramos Kubernetes

Creamos la aplicación con Python

```
alumno02@A6Alumno02: ~/kubernetes-aws-practice$ cat > app.py << 'EOF'
> #!/usr/bin/env python3
from flask import Flask, jsonify, send_from_directory
import os
import socket
from datetime import datetime
import sys
app = Flask(__name__)
# Variables de entorno inyectadas por Kubernetes
POD_NAME = os.getenv('POD_NAME', 'Unknown Pod')
POD_NAMESPACE = os.getenv('POD_NAMESPACE', 'default')
@app.route('/')
def index():
    return send_from_directory('.', 'index.html')
@app.route('/pod-info')
def pod_info():
    return jsonify({
        'pod_name': POD_NAME,
        'namespace': POD_NAMESPACE,
        'hostname': socket.gethostname(),
        'timestamp': datetime.now().isoformat()
    })
@app.route('/health')
def health():
    return jsonify({'status': 'healthy', 'pod': POD_NAME}), 200
if __name__ == '__main__':
    print(f"[{POD_NAME}] Iniciando servidor Flask...", file=sys.stderr)
    app.run(host='0.0.0.0', port=5000, debug=False)
EOF
alumno02@A6Alumno02: ~/kubernetes-aws-practice$ sudo chmod +x app.py
[sudo] password for alumno02:
alumno02@A6Alumno02: ~/kubernetes-aws-practice$
```

Creamos el archivo HTML Y Creamos los requerimientos

Tambien creamos el Configmap y comprobamos kubectl

```
alumno02@A6Alumno02:~/kubernetes-aws-practice$ cat > configmap.yaml << 'EOF'
apiVersion: v1
> apiVersion: v1
kind: ConfigMap
```

```
alumno02@A6Alumno02:~/kubernetes-aws-practice$ kubectl get namespaces
```

| NAME            | STATUS | AGE   |
|-----------------|--------|-------|
| default         | Active | 5h29m |
| kube-node-lease | Active | 5h29m |
| kube-public     | Active | 5h29m |
| kube-system     | Active | 5h29m |

Creamos el Deployment con 3 replicas

```
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ cat > deployment.yaml << 'EOF'
> apiVersion: apps/v1
kind: Deployment
metadata:
name: web-app
namespace: load-balancer-demo
labels:
app: web-app
spec:
replicas: 3
```

El Configyaml tuve que volver ha hacerlo con nano, según ChatGPT me daba error por los saltos, una vez hecho con nano no da errores.

Despues probamos que todo vaya bien

```
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl apply -f configmap
.yaml
configmap/app-files configured
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get configmap -n l
oad-balancer-demo
```

| NAME             | DATA | AGE |
|------------------|------|-----|
| app-files        | 3    | 31m |
| kube-root-ca.crt | 1    | 31m |

```
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl apply -f deploymen
t.yaml
```

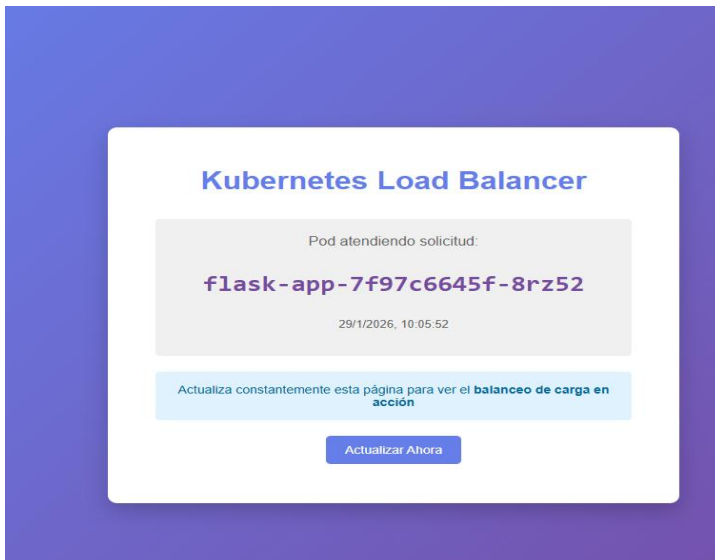
Creamos el servicio Balanceador y comprobamos que todo funcione

```
- protocol: TCP
  port: 80
  targetPort: 5000
  name: http
  sessionAffinity: None
> EOF
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl apply -f service.yaml
service/web-app-service created
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get svc -n load-balancer-demo
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
web-app-service                    LoadBalancer        10.43.205.78     <pending>        80:32512/TCP     9s
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get svc -n load-balancer-demo web-app-service -o wide
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
SELECTOR
web-app-service                    LoadBalancer        10.43.205.78     <pending>        80:32512/TCP     14s
app=flask-app
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$
```

## Parte 3 : Verificar Balanceo de Carga Local

```
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl port-forward -n load-balancer-demo svc/web-app-service 8080:80
Forwarding from 127.0.0.1:8080 -> 5000
Forwarding from [::1]:8080 -> 5000
^Calumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ for i in {1..10}; do
> echo "Petición $i:"
curl -s http://localhost:8080/pod-info | python3 -m json.tool | grep pod_name
sleep 1
done
Petición 1:
Expecting value: line 1 column 1 (char 0)
Petición 2:
Expecting value: line 1 column 1 (char 0)
Petición 3:
Expecting value: line 1 column 1 (char 0)
Petición 4:
Expecting value: line 1 column 1 (char 0)
Petición 5:
Expecting value: line 1 column 1 (char 0)
Petición 6:
Expecting value: line 1 column 1 (char 0)
Petición 7:
Expecting value: line 1 column 1 (char 0)
Petición 8:
Expecting value: line 1 column 1 (char 0)
Petición 9:
Expecting value: line 1 column 1 (char 0)
Petición 10:
Expecting value: line 1 column 1 (char 0)
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$
```

Probamos el navegador (tuve que hacer un par de cambios en el puerto 8080 porque por alguna razón estaba cerrado, y además creo que había un proceso en marcha...



Después de esto pasamos a la Parte 4.

#### Parte 4 CONFIGURAMOS AWS

Empezamos creando el grupo de seguridad

### Detalles básicos

**Nombre del grupo de seguridad** [Información](#)

kubernetes-aws-sg

El nombre no se puede editar después de su creación.

**Descripción** [Información](#)

Security group para Kubernetes con AWS

Creamos la instancia en AWS

### Nombre y etiquetas

[Información](#)

**Nombre**

kubernetes-test-server

Y ahora creamos las reglas de “entrada”

|                                             |                                       |                      |
|---------------------------------------------|---------------------------------------|----------------------|
| Tipo <a href="#">Información</a>            | Protocolo <a href="#">Información</a> | Intervalo            |
| SSH                                         | TCP                                   | 22                   |
| Tipo de destino <a href="#">Información</a> | Destino <a href="#">Información</a>   | Descripción          |
| Personalizada                               | <input type="text"/>                  | <input type="text"/> |

Regla de salida 3

|                                             |                                       |                      |
|---------------------------------------------|---------------------------------------|----------------------|
| Tipo <a href="#">Información</a>            | Protocolo <a href="#">Información</a> | Intervalo            |
| HTTP                                        | TCP                                   | 80                   |
| Tipo de destino <a href="#">Información</a> | Destino <a href="#">Información</a>   | Descripción          |
| Personalizada                               | <input type="text"/>                  | <input type="text"/> |

Regla de salida 4

|                                  |                                       |           |
|----------------------------------|---------------------------------------|-----------|
| Tipo <a href="#">Información</a> | Protocolo <a href="#">Información</a> | Intervalo |
| HTTPS                            | TCP                                   | 443       |

## ▼ Tipo de instancia [Información](#) | [Obtener asesoramiento](#)

### Tipo de instancia

t3.micro

Apto para la capa gratuita

Familia: t3 2 vCPU 1 GiB Memoria Generación actual: true

Bajo demanda Ubuntu Pro base precios: 0.0139 USD por hora

Bajo demanda SUSE base precios: 0.0104 USD por hora

Bajo demanda Linux base precios: 0.0104 USD por hora

### Firewall (grupos de seguridad) [Información](#)

Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas a la instancia.

☐ Crear grupo de seguridad

☒ Seleccionar un grupo de seguridad existente

### Grupos de seguridad comunes [Información](#)

Seleccionar grupos de seguridad

kubernetes-aws-sg sg-086501916da3d47f9 ✕

VPC: vpc-09c6a10b86b904bde

Los grupos de seguridad que agrega o elimina aquí se agregarán a todas las interfaces de red o se eliminarán

Bueno si has mirado bien las capturas te daras cuenta que he puesto reglas de salida, si efectivamente no se leer, bueno he cambiado las reglas de salida por las de entrada y magicamente ahora si me conecta a AWS (normal que no me funcionara el SSH, el puerto no estaba abierto) una vez cambiadas las reglas probamos a conectarnos.



```
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.
```

```
ubuntu@ip-172-31-79-164:~$ sudo apt update
```

```
Hit:1 http://us-east-1-ec2.archive.ubuntu.com/ubuntu noble InRelease
```

#### ▼ Reglas de entrada

| Nombre | ID de la regla del grupo ... | Intervalo de p... | Protocolo | Origen    |
|--------|------------------------------|-------------------|-----------|-----------|
|        | sgr-0c590cfc7d93f223f        | 22                | TCP       | 0.0.0.0/0 |
|        | sgr-057b6a5ab83b1ebed        | 443               | TCP       | 0.0.0.0/0 |
|        | sgr-05c30ab470830ce5e        | 80                | TCP       | 0.0.0.0/0 |

Ahora una vez conectados instalamos las herramientas pertinentes

```
ubuntu@ip-172-31-79-164:~$ sudo apt install -y curl wget python3 python3-pip git  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
curl is already the newest version (8.5.0-2ubuntu10.6).  
curl set to manually installed.  
wget is already the newest version (1.21.4-1ubuntu4.1).
```

Despues probamos a crear el directorio de trabajo

```
ubuntu@ip-172-31-79-164:~$ mkdir -p ~/kubernetes-test  
ubuntu@ip-172-31-79-164:~$
```

Ahora abrimos en paralelo otro WSL

```
^[A^Calumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ ssh -i ~/.ssh/labsuser.pem -N -R 8888:localhost:8080 ubun  
tu@100.28.227.211
```

Aqui tenemos la 3º terminal

```
ubuntu@ip-172-31-79-164:~$ curl -s http://localhost:8888/pod-info  
{  
  "hostname": "flask-app-7f97c6645f-8rz52",  
  "namespace": "load-balancer-demo",  
  "pod_name": "flask-app-7f97c6645f-8rz52",  
  "timestamp": "2026-02-03T08:50:18.554227"  
}  
ubuntu@ip-172-31-79-164:~$
```



Ahora creamos el script en la instancia EC2

```
echo ""
echo "=== Resumen Balanceo ==="
for pod in "${!pods_count[@]}"; do
    echo "Pod $pod: ${pods_count[$pod]} peticiones"
done
> EOF
ubuntu@ip-172-31-79-164:~$ sudo chmod +x ~/test-kubernetes-lb.sh
ubuntu@ip-172-31-79-164:~$ ~/test-kubernetes-lb.sh
=== Prueba Kubernetes Load Balancer desde AWS ===

Petición 1 → Pod: flask-app-7f97c6645f-8rz52
Petición 2 → Pod: flask-app-7f97c6645f-8rz52
Petición 3 → Pod: flask-app-7f97c6645f-8rz52
Petición 4 → Pod: flask-app-7f97c6645f-8rz52
Petición 5 → Pod: flask-app-7f97c6645f-8rz52
Petición 6 → Pod: flask-app-7f97c6645f-8rz52
^C
ubuntu@ip-172-31-79-164:~$
```

## PARTE 6 Hacemos el escalado

```
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKU6$ kubectl scale deployment flask-app -n load-balancer-demo --replicas=5
deployment.apps/flask-app scaled
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKU6$ kubectl get pods -n load-balancer-demo
NAME                                READY   STATUS    RESTARTS   AGE
flask-app-7f97c6645f-7zfw6          1/1     Running   0           3s
flask-app-7f97c6645f-8njms          1/1     Running   0           3s
flask-app-7f97c6645f-8rz52          1/1     Running   2 (96m ago) 4d23h
flask-app-7f97c6645f-wzg99          1/1     Running   2 (96m ago) 4d23h
flask-app-7f97c6645f-zf4tn          1/1     Running   0           3s
```

Y esperamos a que este ready

```
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKU6$ kubectl wait --for=condition=ready pod -l app=flask-app -n load-balancer-demo --timeout=120s
pod/flask-app-7f97c6645f-7zfw6 condition met
pod/flask-app-7f97c6645f-8njms condition met
pod/flask-app-7f97c6645f-8rz52 condition met
pod/flask-app-7f97c6645f-wzg99 condition met
pod/flask-app-7f97c6645f-zf4tn condition met
```

Probamos el balanceo

```
ubuntu@ip-172-31-79-164:~$ ~/test-kubernetes-lb.sh
=== Prueba Kubernetes Load Balancer desde AWS ===
```

# PARTE 7 Monitoreo y Observabilidad

Comprobamos los pods

```
aLumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl get pods -n load-balancer-demo
NAME                                READY   STATUS    RESTARTS   AGE
flask-app-7f97c6645f-7zfw6         1/1     Running   0           27m
flask-app-7f97c6645f-8njms         1/1     Running   0           27m
flask-app-7f97c6645f-8rz52         1/1     Running   2 (124m ago)  4d23h
flask-app-7f97c6645f-wzg99         1/1     Running   2 (124m ago)  4d23h
flask-app-7f97c6645f-zf4tn         1/1     Running   0           27m
aLumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl logs -n load-balancer-demo flask-app-7f97c6645f-7zfw6
Collecting Flask==3.0.0 (from -r /app/requirements.txt (line 1))
  Downloading flask-3.0.0-py3-none-any.whl.metadata (3.6 kB)
Collecting Werkzeug==3.0.0 (from -r /app/requirements.txt (line 2))
  Downloading werkzeug-3.0.0-py3-none-any.whl.metadata (4.1 kB)
Collecting Jinja2>=3.1.2 (from Flask==3.0.0->-r /app/requirements.txt (line 1))
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting itsdangerous>=2.1.2 (from Flask==3.0.0->-r /app/requirements.txt (line 1))
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting click>=8.1.3 (from Flask==3.0.0->-r /app/requirements.txt (line 1))
  Downloading click-8.1.3-py3-none-any.whl.metadata (2.6 kB)
```

En este caso cogi el primer POD y lo comprobamos

```
aLumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl logs -n load-balancer-demo -l app=flask-app -f
[notice] A new release of pip is available: 24.0 -> 26.0
[notice] To update, run: pip install --upgrade pip
[flask-app-7f97c6645f-7zfw6] Iniciando servidor Flask...
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server inst
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.42.0.39:5000
Press CTRL+C to quit
127.0.0.1 - - [03/Feb/2026 08:56:39] "GET /pod-info HTTP/1.1" 200 -
[notice] A new release of pip is available: 24.0 -> 26.0
[notice] To update, run: pip install --upgrade pip
```

Ahora comprobamos CPU y memoria de pods

```
aLumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl top pods -n load-balancer-demo
NAME                                CPU(cores)   MEMORY(bytes)
flask-app-7f97c6645f-7zfw6         1m           24Mi
flask-app-7f97c6645f-8njms         1m           24Mi
flask-app-7f97c6645f-8rz52         1m           28Mi
flask-app-7f97c6645f-wzg99         1m           34Mi
flask-app-7f97c6645f-zf4tn         1m           24Mi
aLumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl top nodes
NAME           CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
a6alumno02    200m        1%       951Mi           12%
aLumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl get events -n load-balancer-demo
LAST SEEN   TYPE      REASON          OBJECT                                     MESSAGE
42m         Normal    Scheduled       pod/flask-app-7f97c6645f-7zfw6          Successfully assigned load-balancer-demo/flask-app-7f97c6645f-7zfw6 to a6alumno02
42m         Normal    Pulled          pod/flask-app-7f97c6645f-7zfw6          Container image "python:3.11-slim" already present on machine
42m         Normal    Created        pod/flask-app-7f97c6645f-7zfw6          Created container: flask-container
42m         Normal    Started        pod/flask-app-7f97c6645f-7zfw6          Started container flask-container
42m         Normal    Scheduled       pod/flask-app-7f97c6645f-8njms          Successfully assigned load-balancer-demo/flask-app-7f97c6645f-8njms to a6alumno02
```

Vemos detalles del deployment

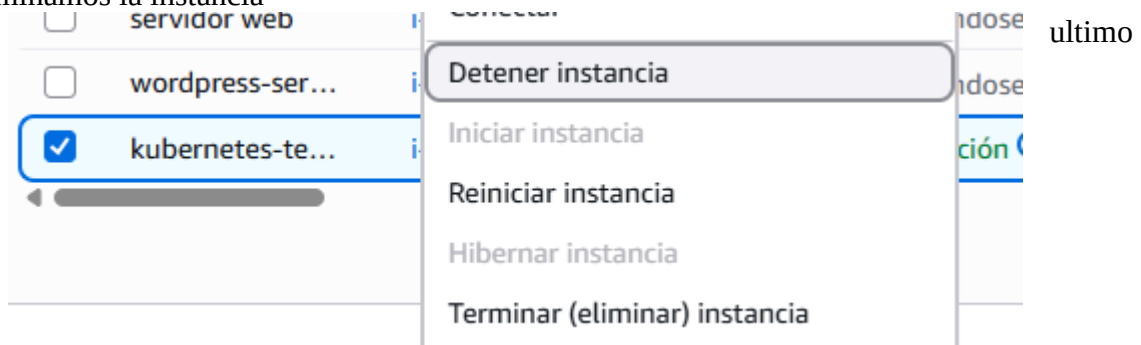
```
aLumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl describe deployment flask-app -n load-balancer-demo
Name: flask-app
Namespace: load-balancer-demo
CreationTimestamp: Thu, 29 Jan 2026 10:00:03 +0100
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 3
Selector: app=flask-app
Replicas: 5 desired | 5 updated | 5 total | 5 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  containers:
    flask-app
    image: python:3.11-slim
    ports:
      - containerPort: 5000
  service: flask-app
aLumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DISKUG$ kubectl rollout history deployment flask-app -n load-balancer-demo
deployment.apps/flask-app
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>
```

## Parte 8 Eliminamos los recursos

```
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl delete namespace load-balancer-demo
namespace "load-balancer-demo" deleted
^Calumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ kubectl get namespaces
NAME                STATUS    AGE
default             Active   7d2h
kube-node-lease     Active   7d2h
kube-public         Active   7d2h
kube-system         Active   7d2h
load-balancer-demo  Terminating 5d1h
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$
```

Ahora eliminamos la instancia

Y por



pausamos k3s

```
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo systemctl stop k3s
[sudo] password for alumno02:
alumno02@A6Alumno02:/mnt/c/Users/Alumno.DESKTOP-DI5KTUG$ sudo /usr/local/bin/k3s-uninstall.sh
+ id -u
+ [ 0 -eq 0 ]
+ K3S_DATA_DIR=/var/lib/rancher/k3s
+ /usr/local/bin/k3s-killall.sh
+ [ -s /etc/systemd/system/k3s.service ]
+ basename /etc/systemd/system/k3s.service
+ systemctl stop k3s.service
```

Y con esto hemos terminado la practica, la verdad es que me la esperaba mas compleja a excepcion de volver a usar AWS (se me habia olvidado como iba) y conectarme otra vez con la clave me daba problemas por poner reglas de salida en lugar de entradas, tambien la parte de las 3 terminales es un poco bizarra pero me ha gustado mucho la practica, muy interesante!!.